

Yusuf Shahzad
ITAI 1378 - Introduction to AI and Intelligent Applications
January 29, 2026

Reflection Journal: Image Processing Lab

Section 1: Technical Understanding

The most surprising discovery about how images are represented was realizing that every photograph, no matter how complex or beautiful, is fundamentally just a three-dimensional array of numbers. When I first saw the $200 \times 200 \times 3$ matrix shape and the explicit 0–255 range in the lab, it transformed my understanding from thinking of images as continuous visual experiences to recognizing them as discrete numerical data structures. The fact that 120,000 bytes of memory could encode all the color information, and that this could be cleanly separated into three overlapping grayscale layers (red, green, and blue channels), made the abstract concept of digital imaging tangibly concrete. Seeing actual pixel values printed on a 20×20 patch—just integers that our brains interpret as colors—was a profound moment that demystified digital photography.

The mathematical operations we implemented translate directly to the visual effects we see every day. Point operations like brightness and contrast adjustments demonstrate how simple arithmetic (adding a constant or multiplying by a scalar) creates the familiar controls in photo editing apps. When we increased brightness by +50, the entire histogram shifted right without changing the image's spatial structure—each pixel moved independently. Neighborhood operations revealed something more profound: that visual effects like blurring, sharpening, and edge detection emerge from local relationships between pixels. The convolution operation, where we multiply small kernels (3×3 matrices) with image patches, is the mathematical engine behind nearly every filter. Seeing how the Sobel operator's horizontal and vertical kernels extract edges by emphasizing intensity differences made it clear that "sharpness" and "blur" aren't mystical properties but predictable outcomes of weighted averaging and differentiation.

The most challenging technique to understand was the perspective transform and its underlying mathematics. While point operations and even convolution filters have intuitive geometric interpretations, perspective warping required grasping projective geometry—the idea that four corner points could define a complete transformation matrix that maps every pixel in a non-linear way. Understanding how `PerspectiveTransform` computes this matrix from just eight numbers (four x,y pairs) was intellectually demanding because it involves solving a system of linear equations that accounts for the "vanishing point" effect we see in real-world photography. The challenge was worth it, though, because it revealed how document scanners automatically rectify skewed photos and how augmented reality systems overlay graphics onto tilted surfaces.

Section 2: Connections and Applications

Today's lab activities connected powerfully to the Nano Banana demonstration from class, showing that AI-powered tools are built on the same mathematical foundations we implemented manually. In class, Nano Banana could apply complex artistic styles to images with a single text prompt, seemingly "understanding" abstract concepts like "vintage" or "dramatic." After today's lab, I understand that these AI models are fundamentally just using many layers of learned convolution filters—the same 3×3 and 5×5 kernels we hand-coded, but discovered automatically through training on millions of images. When we created our "vintage" filter by adjusting color channels and reducing contrast, we were essentially replicating what a style-transfer network does, except the AI learns the optimal filter parameters instead of us designing them manually. The edge detection and texture extraction we performed are primitive versions of what convolutional neural networks do in their early layers. Nano Banana's "magic" is really just these classical operations stacked deeply and trained to recognize which combinations produce particular aesthetic effects.

The real-world applications for these techniques are vast and often invisible in our daily lives. Medical imaging relies heavily on histogram equalization and edge detection—the same operations we implemented—to help radiologists identify tumors in low-contrast X-rays and MRI scans. Autonomous vehicles use Sobel and Canny edge detectors to identify lane markings, traffic signs, and obstacles in real-time, even in challenging lighting conditions where CLAHE preprocessing improves detection rates. Document scanning apps use perspective transforms to automatically rectify photos of receipts, whiteboards, and textbook pages, making them readable for OCR systems. Even smartphone cameras apply these classical filters: every time someone takes a portrait-mode photo, the phone is using edge detection to separate subject from background, and every Instagram filter is a carefully tuned combination of the color-channel manipulations and contrast adjustments we explored.

For a future project combining traditional and AI approaches, I envision a hybrid medical imaging analysis system. The traditional preprocessing pipeline would use histogram equalization and adaptive thresholding to normalize X-ray images from different machines and exposure settings, then apply Canny edge detection to identify potential regions of interest. These preprocessed regions would feed into a deep learning model fine-tuned for specific pathology detection. The advantage of this hybrid approach is interpretability: doctors can understand and trust the preprocessing steps because they're deterministic and visualizable, while still benefiting from AI's pattern-recognition strengths for the final diagnosis. This mirrors how radiologists actually work—they don't just look at raw images; they mentally apply contrast adjustments and focus on edges and textures, which we can now automate systematically before invoking AI.

Section 3: Personal Reflection

The aspect of image processing that interests me most for further exploration is frequency domain processing using Fourier transforms. During the lab, all our operations happened in the spatial domain—we manipulated pixels based on their positions and neighbors. But I learned that images can also be represented as combinations of sine waves at different frequencies, and that operations like removing periodic noise patterns or perfectly separating "detail" from "structure" are much easier in the frequency domain. This connects to signal processing and has applications in medical imaging (CT and MRI reconstruction), image compression (JPEG uses discrete cosine transforms), and even forensics (detecting image tampering through frequency analysis). I'm curious to implement a Fourier-based denoising filter and compare its results to the spatial filters we used today.

This lab has fundamentally changed my understanding of digital photography and image editing. Before, I thought of photo editing tools as applying "effects" that were somewhat magical or artistic. Now I understand that every slider in Lightroom, every filter in Instagram, every enhancement in Google Photos is implementing the same mathematical operations we coded today—they're just packaged in user-friendly interfaces. When a photographer talks about "bringing out detail" in shadows, I now know they're stretching the histogram and applying local contrast enhancement, essentially our CLAHE implementation. When someone says a photo looks "sharp," I understand they're perceiving high-frequency information that could be enhanced with an unsharp masking kernel. This knowledge doesn't diminish the artistry of photography; instead, it deepens my appreciation for how technical understanding enables creative expression.

Despite the comprehensive lab experience, I still have questions about image processing. How do modern machine learning models decide which filters to learn in each layer, and is there a way to interpret or visualize what high-level features they've discovered beyond the low-level edges we extracted? How do video codecs efficiently compress temporal information—is it just applying these operations frame-by-frame, or are there fundamentally different techniques? And practically, how do professional tools like Photoshop implement these operations so quickly on high-resolution images—are they using GPU acceleration, and if so, how does parallelizing convolution operations work at the hardware level? These questions motivate me to explore both the theoretical computer vision literature and the practical software engineering aspects of real-world image processing systems.